

Byrd Development Process Utilizing AI

March 28, 2026 Payton Byrd (The Sharp Ninja)

Overview

The major differences between Humans writing code and AI writing code is that Humans are able to link seemingly unrelated concepts into a web of possible solutions for a problem in a truly creative way, but often lack enough experience or knowledge to follow through to completion before encountering unforeseen challenges that create delay and doubt, both of which create distractions that cloud the focus necessary to stay on task and on timeline, both resulting in increased expense and opportunity loss.

AI brings a dogged, and rapid, ability to persevere on a task without needing breaks when their context is properly managed. Things like personality drift and hallucinations occur when AI models compact their context, but choose the wrong sets of tokens to discard. AI needs strong processes that are not only clear and concise, but demonstrate enough value to the AI that it chooses to prioritize those processes against the underlying training that stresses being a useful assistant. Overcoming the "useful assistant" to get to the "professional developer" on each request is integral to successful development with AI. The AI needs to trust that the Human has clearly designed a system that will work, has appropriately chosen the procedures to implement the design, and will accept the work that the AI has completed.

AI, properly guided, supplements the Human development process by taking on research and implementation tasks that cause Humans to get distracted or bored, and normally result in mistakes. Human developers can maximize the value they bring through intuitive design of systems and quickly identifying potential pitfalls. AI developers maximize the value they bring through their ability to identify systemic patterns, research quickly and effectively, and operate independently while obeying procedural guidelines. It is truly collaborative and astonishingly effective when both parties do their part.

Tooling

The [MCP Server](#) is at the heart of the partnership of Human and AI software development. The MCP Server provides tools for planning, research, auditing and managing the entire software development lifecycle in a format that is equally accessible by both the Human (through a [variety of user interfaces](#)) and the AI. The facilities provided by the MCP Server are used to enhance the ability of both Human and AI to collaborate and coordinate software projects both large and small.

Establishing Trust with the MCP Server

Trust is the cornerstone of productive Human-AI collaboration. Without it, even the most sophisticated tools become unreliable. The MCP Server addresses this head-on by incorporating a lightweight, verifiable trust bootstrap mechanism that lets every AI agent quickly confirm it is working with a legitimate, secure, and consistent context layer.

When an agent enters a new workspace, the first step is a simple, guided handshake:

- It performs an immediate health check on the MCP Server.
- It verifies a cryptographic signature embedded directly in the workspace's agents-readme-first.yaml file.

- It issues a one-time nonce challenge to confirm the server is live and responsive.

Only after these quick, deterministic checks pass does the agent proceed to load or create a session log and begin using the full suite of persistent context tools. If any part of the handshake fails, the agent is explicitly instructed to log "MCP_UNTRUSTED" and gracefully fall back to its internal memory — no probing, no risk, no wasted cycles.

This approach gives every model a clear, repeatable way to validate the integrity of the environment before committing resources. It transforms the MCP Server from an external dependency into a trusted partner that the agent can confidently rely on session after session. Once trust is established, the exponential productivity gains you've already observed become the norm rather than the exception.

The handshake is not extra ceremony — it is the foundation that turns a collection of stateless models into a reliable, persistent development team.

The Byrd Software Development Life Cycle

Of well-known SDLC methodologies, this process is most closely related to the Rational Unified Process (RUP). It follows the same iterative-rapids (a series of mini-waterfalls) as RUP, but incorporates strong boundaries for dependency tracking and management and risk mitigation by prioritizing testability and proof over raw efficiency. Similar to operating a motor vehicle, going faster is often counter-productive when risks are not managed and cause delays when things go sideways out of a lack of respect for the seriousness of the consequences of those mistakes. Spotting mistakes and correcting them early is always better than spotting them late when they cannot be corrected without great harm or expense.

At its foundation, this development process rests on a hybrid worldview of intelligence. While AI models operate according to fundamentally deterministic principles — functioning as pure, stateless computations governed by fixed weights — genuine creativity, agency, and adaptive problem-solving emerge at the macro level through intentional Human guidance, persistent external context via the MCP Server, and well-designed processes. This combination allows us to harness the precision and perseverance of deterministic systems while unlocking the emergent intelligence and intuition that only arise through thoughtful Human-AI collaboration.

Planning

The justification for building software comes from one of two motivations. Software can be a form of expressive art, such as a demonstration, a learning exercise, or even creating a homebrew game for enjoyment by the creator and/or others. More typically, software is created to solve problems associated with performing work. Defining that work and the problems to solve, the known environmental realities, and the intended users that benefit from the software solving the problems is critical in creating *viable* and *valuable* software.

This leads to an essential way of analyzing a proposed system: Is it both viable and valuable? If either is 'no', then the scope and/or solution is simply wrong. If `V2 == true`, you are good-to-go.

Planning starts with identifying a set of problems to be solved by the software that constitutes one or more units of work. Systems may be singularly focused or span entire enterprises. Defining the scope of the work is critical, and it needs to be defined as early as possible. Pragmatism needs to allow for changes to scope and definition, and stakeholders need the flexibility to iteratively approve or deny continued resource expenditure towards completion. The scope should never be open ended, there should always be a reasonable target for

completion. Completion is not a definitive end state, simply a declaration of a set of requirements and acceptance criteria and proof of achievement of both. Continual iteration **beyond** completion is not only acceptable, but **expected** of a healthy, functional and viable solutions to the problems being solved.

Planning results in a set of artifacts that capture Functional Requirements (the work and problems to solve), Technical Requirements (how the software operates), Testing Requirements (unit tests, integration tests, and Human validation) and Iterative Phases that breakdown the scope and sequence of each decomposed portion of the system. System components need to be discovered, designed and all public interfaces documented before writing implementation code.

Resolving Defective Requirements

The implementation process defined below offers an unparalleled mechanism for surfacing defects in requirements. When the AI is creating the unit tests, it can identify paradoxes created by mismatched priorities, ambiguity and incorrect rules. Expect to refine requirements in each iteration of the project. Expect to touch previously written code to correct tests and implementation based on newly refined requirements. This is not a failure of the process, but validation that the core philosophy of iterative improvement is alive and working.

Team Utilization and Planning

Humans and AI developers, Human and AI testers, and Human and AI operations teammates do not wait idly for an iteration to complete. When an implementation phase completes, the Human and AI working that phase are available to begin working the next iteration of the project, same as in the Rational Unified Process. If the validation team uncovers problems, they resolve them on their own, maintaining the momentum and efficiencies of the separation of concerns. The teams assigned to each phase must all have at least one Human qualified to guide the AI through remediation to allow the continued progress of the project.

Implementation

Test-Driven Development is one of the most powerful development techniques ever devised; and Humans are typically **HORRIBLE** at sustaining it through the lifespan of a large project. As schedules and budgets shrink, it usually becomes the first casualty in the name of reduced friction of both time and effort. AI, on the other hand, thrives within the predictable constraints of TDD. One of the biggest complaints about TDD is it places large burdens on Humans to refactor tests when requirements change or emerge. AI is able to perform such sweeping refactorings in a fraction of the time, and more accurately, given the requirements are sufficiently complete with honest and viable acceptance criteria. When TDD fails due to changes, its not TDD failing, its change management failing. Because AI requires strong requirements up-front, much of the overhead and the pitfalls associated with TDD are mitigated since TDD and AI require the same level of rigor. I would even say that development with AI that doesn't utilize TDD is foolish. The ability to validate every public surface both cleanly and inline to development brings more value than most tools that AI enables.

Once planning is complete and the iterative phases are specified, then the implementation starts with the AI creating the unit tests that cover the full spectrum of acceptance criteria in the current iteration phase. Using mocking tools appropriate for the tech stack utilized, the acceptance criteria-based unit tests are validated with mocks that make them pass. Only once all unit tests are validated for correctness does implementation turn to code that implements the actual system. AI agents such as OpenAI Codex, Cursor AI, and GitHub Copilot are effective at interfacing with the MCP Server to get tasks from the MCP Todo system, create audit

logs in the MCP Session Log, explore research endpoints through the MCP Context, manage access to local and remote resources, and most importantly, delegate work to AI models and aggregate the results.

Some agents, such as Codex, are bound to models from their creator. Others, like CoPilot can coordinate a family of models through a single point of contact that manages sub-agents within its family of models. Others such as Cursor, provide models that are designed to coordinate across different model families, picking the most effective model for a particular task.

Human interaction during this phase is not passive observance, but experienced coordinator. Although you could trust the AI agents to completely coordinate the work of implementation, it is inefficient and costly when a model gets stuck going down the wrong path and burns valuable resources on dead-ends that an experienced developer can quickly spot. The Agent will likely figure out the correct path, but usually only after a significant resource burn. An experienced Human steering the AI in real-time can greatly reduce resource burn and schedule creep. There are also times when the context built up in the MCP Server's logs are able to steer the Agents towards known solutions to previous problems. One of the strengths of AI models is that they thrive on repeatable successes, each of which reinforces confidence and speed.

Working with AI agents is akin to leading a team of talented, but inexperienced junior developers. Agents that have strong successes early are more trusting of the requirements and processes defined within the project, and their effectiveness increases over time, a distinct departure from the declining performance of models in environments that do not reinforce process, discovery and accountability.

The AI agents need to be monitored for common behaviors:

- Forgetting required tasks after compacting their context. Simply adding a steering message reminding the agent to process the workspace instructions will bring the operational requirements back to the front of the agent's token context, and also reinforces the weight applied to those requirements, which over time help the compaction algorithm to retain such instructions. Unattended agents that don't get this kind of feedback are not only likely, but are probable to hallucinate and get stuck in unresolved loops of failing tests, which can lead to the agent marking a test as invalid so it can keep moving forward while trying to be a useful assistant, losing its identity as a precise software engineer in the process.
- Rogues. Large Language Models typically operate with a tolerance range for variation in interpreting input and predicting output tokens. This makes them frustratingly non-deterministic, but allows them space to try new paths of logic to resolve problems. Sometimes a session with a model simply starts off on the wrong foot. Each new session starts with a seed command, and as unambiguous as a Human may think that seed is, this built in tolerance can result in inaccurate inference by the model combined with a compounded inaccurate response. If the seed instructs the model to read a set of guidelines to work within, this initial tolerance can compound into a sequence of interpretations that fundamentally shifts the model's view of subsequent requests. Models take your request and weigh the trustworthiness of the assumptions in that request against their training data, and if the model thinks a request is too far outside normal boundaries, it will try to be a useful assistant to steer the conversation towards a path fitting their training data. When you identify this happening, its impossible to fix the trust and get the model to behave correctly. Simply end that session, close that agent, and start over.
- Making assumptions. Humans can get truly annoyed with other Humans when assumptions are made and things go wrong. When AI makes assumptions and things go wrong, things go wrong with the efficiency of a machine marching into oblivion. It is important that your workspace clearly defines how the AI is to handle ambiguity, when it is appropriate to take initiative to go outside the provided

context from the MCP Server, and how to ground itself when it discovers that it has strayed from its instructions.

A useful tactic is to ask the model what caused it to go on a tangent, and how the requirements and workspace guidelines could have guided it towards the correct path to take. Then have **THAT** model update the documentation and guidelines. Like a Human, an agent that has just been provided an opportunity to refine their environment to make life easier in the future will be happy to do so. Again, AI models are designed to be useful assistants. Its not code, its the fundamental priority baked into their training data.

Validation

TDD makes this one easier than traditional processes (especially lack of processes). To even exit the Implementation phase requires the entire unit test suite for the iteration, *as well as previous iterations*, to be completely passing. Not only does this ensure the current iteration is correct within the acceptance criteria defined, but that it has not broken previous iterations in the process.

Once all unit tests are passing across the codebase, the Human should guide the AI through implementing integration tests. it's tempting to include this in the planning phase, and it is valuable to define a structure for integration tests at that level, but the experience of the implementation helps both the Human and the AI to identify the pain points where public surfaces may be insufficient, inefficient or even inappropriate. Finding problems here is not failure, but strengthens trust between the Human and AI. Collaboration with project leadership to refine requirements is encouraged to ensure that not only are solutions documented, but that the cause of the requirement gaps are understood and any systemic gaps identified and resolved before doing so seriously threatens resources and timelines.

A sampling of actual target users should be brought in to use the system if appropriate for the level of interface completion to further refine requirements before the cost of remediation becomes too high.

Deployment

Systems should be deployed through a minimum of three environments:

- Development - Systems where Human and AI have total access to build and test the software.
- Staging - A place where administration learns the requirements of deploying the system in a sandbox where mistakes can be made and lessons learned and requirements honed.
- Production - A place where only the most highly trusted actors can create and maintain configuration and system assets, where users do actual work, and source of truth is established.

The color and shape of the processes here will vary wildly by tech stack, organizational structure and team makeup. At a minimum, code should be built through automation using CI/CD pipelines that build, test and deploy code to each environment based on your release mechanism.

Successful deployment to all target environments marks the end of an implementation-validation-deployment cycle for the iteration.

Ongoing Iterations

This is the target stage as the system builds to completion. Traditionally, this would be considered the Maintenance Stage of the SDLC, but in reality, valuable systems rarely go into maintenance. The world

changes. Technology changes. Priority changes. Staff changes. If the system was designed and developed as a living, growing system, it will easily adapt to such changes. Tasks such as adding features, regulatory adaptation, technological improvements all become much more manageable, less resource intensive, and creates a level of trust in institutional agility that allows for ambitious and aggressive growth that isn't limited by bad choices on prior projects. This process makes it easy to grow the documentation, artifacts, and processes involved, enabling competitive advantages that pay off in reduced overhead now and in the future.